

Learning to Place Guards by Reinforcement: A Geo-Free Neural Policy for the Vertex-Guard Art Gallery Problem

Domagoj Ševerdija^{a,*}, Jurica Maltar^a, Nathan Chappel, Domagoj Matijević^a

^a*School of Applied Mathematics and Informatics, University J. J. Strossmayer of Osijek,
Croatia*

Abstract

Neural combinatorial optimization (NCO) has shown that policies trained by reinforcement can construct strong solutions to NP-hard problems directly from raw instances. What such a policy actually *learns*, as opposed to what its decoder *expresses*, remains much less clear. We study this distinction on the vertex-guard Art Gallery Problem, the NP-hard task of choosing polygon vertices from which to observe an entire region. A pointer-network policy is trained from a coverage-aware reward over its own rollouts under the constraint we call *geo-free* inference: at test time it sees only vertex coordinates, with no visibility computation and no geometric oracle. The policy places guards economically but leaves a tail of under-covered polygons that widens far beyond the training range. To locate the cause, we freeze the trained encoder and read its embeddings with a small single-shot classifier, still *geo-free* at inference. The classifier closes most of the feasibility gap, in and out of distribution and at up to roughly five times the training range, cutting under-covered polygons by about an order of magnitude at an explicitly reported cost in guard count. We read this as evidence that the reinforcement-trained representation already encodes the geometry a feasibility decision requires, and that the residual failures reflect decoder calibration, not missing knowledge. Probing a frozen encoder thus offers a practical way to ask what a neural combinatorial solver has internalized.

Keywords: Art Gallery Problem, neural combinatorial optimization, reinforcement learning, pointer networks, representation probing

*Corresponding author

Email addresses: dseverdi@mathos.hr (Domagoj Ševerdija), jmaltar@mathos.hr (Jurica Maltar), nathan.s.chappel@gmail.com (Nathan Chappel), domagoj@mathos.hr (Domagoj Matijević)

1. Introduction

The Art Gallery Problem (AGP) asks for the minimum number of guards needed to observe every point of a simple polygon [1, 2], with applications in surveillance, sensor placement, and robotics [3, 4]. We focus on the *vertex-guard* variant, in which guards must be chosen from the polygon’s vertices; the problem remains NP-hard and is a discrete instance of geometric set cover with non-uniform, polygon-dependent visibility regions. The question this paper investigates is whether a neural policy can learn to place such guards from coordinates alone, trained from a coverage-aware reward signal over its own rollouts, with no visibility matrix and no geometric oracle at the moment it chooses guards. We are not asking whether a learned solver can match a classical greedy heuristic on guard count — it cannot, and we do not claim it does. We are asking whether the underlying combinatorial-geometric task is learnable when the model is denied any explicit geometric input at test time.

The reinforcement method we evaluate is an LSTM pointer-network policy trained with Preference Optimization (PO) [5] on Bradley–Terry preference pairs [6] drawn from the policy’s own rollouts. The policy emits a vertex-index sequence with an EOS token; the reward couples coverage and guard usage. PO replaces REINFORCE advantages [7, 8] with binary preferences between pairs of rollouts, sidestepping REINFORCE’s variance and the brittleness of supervised cloning of search labels. We refer to inference under this policy as *geo-free* — the network reads only vertex coordinates at the moment it places guards, never a visibility oracle (defined precisely in Section 2.3).

The policy works, to a degree. On a held-out in-distribution split its guard sets are competitive in cardinality with classical baselines, but the per-polygon coverage distribution has a tail: a non-trivial fraction of polygons end up below the feasibility line one would want a deployed solver to clear, and the failure rate grows on out-of-distribution polygons whose vertex counts exceed the training range. The reinforcement method, by itself, is not feasibility-guaranteed.

To check whether this residual reflects a true limit of the reinforcement-learned representation or only a calibration problem in the decoder, we train a small per-vertex classifier that reads only the frozen encoder embeddings of the policy and predicts a vertex-inclusion probability, controlled by a single scalar threshold. The classifier has no access to visibility information at inference, and it compresses the feasibility tail substantially both in and out of distribution. We read this as evidence that the reinforcement-trained encoder already carries enough geometric structure to support a feasibility decision, and that the policy’s coverage tail was a decoder-calibration limit rather than a representation limit. The cost of recovering feasibility is a controlled rise in guard count, which we report explicitly.

Contributions.. This paper makes four contributions.

C1 A representation–decoder decomposition for neural combinatorial optimization. We adapt the probing methodology of representation

analysis [9, 10, 11] to a reinforcement-trained combinatorial solver: freezing the policy’s encoder and training a single-shot per-vertex probe over its embeddings isolates what the learner *represents* from what its decoder *expresses*, with a no-encoder ablation as the control. To our knowledge this is the first such probing study of an RL-trained policy on a geometric covering problem.

C2 Out-of-distribution generalization at scale. On polygons whose vertex counts reach roughly five times the training range, the probe raises the average fraction meeting the 0.95 feasibility gate across four probe seeds from 85% (policy seed) to 99% (Section 6.3).

C3 Geo-free inference as a protocol. Restricting test-time inputs to vertex coordinates isolates what a learner internalizes from what a visibility oracle could compute for it, making “no explicit geometric knowledge” a measurable property rather than a rhetorical one (Section 2.3).

C4 A structural finding on post-processing. Iterative add/remove/stop editors fail to improve a strong seed in any coverage-preserving regime, whereas the single-shot probe is empirically a fixed point under iteration (Section 6.4); the contrast is what makes the single-shot design the one that exposes the encoder’s information.

We do not claim to beat classical greedy or local-search heuristics on guard count; we claim, and document, that vertex-guard placement is learnable to a measurable degree by a reinforcement method that never reads an explicit visibility object at inference.

The remainder of the paper is organized as follows. Section 2 fixes notation and the geo-free constraint. Section 3 reviews related work. Section 4 describes the reinforcement method and the representation probe. Section 5 describes the experimental setup. Section 6 reports the results. Section 7 reads the results against the question. Section 8 states limitations and Section 9 concludes.

2. Problem and Notation

2.1. Polygons and visibility

Let $P \subset \mathbb{R}^2$ denote a simple polygonal region (the closed set consisting of its interior and boundary), with n vertices at coordinates $x_1, \dots, x_n \in \mathbb{R}^2$ and vertex index set $V(P) = \{1, \dots, n\}$. A point p *sees* a point q when the closed segment between them stays inside the polygon, $\overline{pq} \subseteq P$. Guards are placed at vertices: for a vertex-guard set $S \subseteq V(P)$, the visibility region and area-normalized coverage are

$$\text{Vis}(S) = \{q \in P : \exists i \in S, x_i \text{ sees } q\}, \quad \text{Cov}(S) = \frac{\text{Area}(\text{Vis}(S))}{\text{Area}(P)} \in [0, 1]. \quad (1)$$

2.2. The vertex-guard variant

The *vertex-guard Art Gallery Problem (AGPVG)* asks for the smallest vertex-guard set that covers the whole polygon. We define the *vertex-guard optimum* directly:

$$\text{OPT}(P) = \min\{|S| : S \subseteq V(P), \text{Cov}(S) = 1\}. \quad (2)$$

Restricting guards to $V(P)$ reduces placement to a finite discrete choice over n vertices — the natural action space for a pointer-network policy [12, 8, 13, 14] — while retaining the full NP-hardness and non-uniform visibility structure of the problem. The public AGPVG instance library provides vertex-guard optimal solutions; we use the library’s per-instance optima as OPT throughout the paper (see Section 5.1).

Equation (2) is a *constrained* single-objective problem — minimize the guard count subject to the hard constraint $\text{Cov}(S) = 1$ — and is a discrete set-cover instance, where coverage is the submodular union of per-guard visibility regions. This is what separates it from the routing problems on which neural combinatorial solvers are usually studied, where any permutation is feasible and only a single scalar cost is minimized. A learner denied a visibility oracle at inference (the geo-free constraint, Section 2.3) cannot certify the coverage constraint while placing guards; it must instead trade coverage against guard count, so the single constrained objective becomes a two-axis operating curve — which we characterize in Section 6.2 by sweeping a decision threshold.

2.3. Geo-free inference

We will say that an inference procedure is *geo-free* if its inputs at test time are limited to the polygon’s vertex coordinates and learned features derived from them — no polygon visibility matrix, no CGAL-computed visibility regions, no per-vertex area or visibility-fraction feature. Geo-free inference does not preclude using exact visibility during evaluation (to report coverage after the fact) or during training (to compute rewards, gate trajectories, or build supervised targets); it restricts what the model reads at the moment it places guards. A solver allowed to query a visibility oracle at every decision step — as classical local search and active-search decoding both do — can in principle simulate greedy or local search, and a measurement of what such a solver has learned would be conflated with a measurement of what its oracle can compute. The geo-free constraint isolates the first question from the second, which is what makes “no explicit geometric knowledge” a measurable property and not just a rhetorical one.

3. Related Work

This section places the present work in four lines of research: algorithms for the Art Gallery Problem (classical and learned), neural combinatorial optimization with reinforcement learning, supervised post-processing of learned solvers, and probing of learned representations.

The Art Gallery Problem.. Lee and Lin [1] established NP-hardness for several variants of the AGP; O’Rourke [2] surveys the foundational geometric and combinatorial results. A closely related geometric guarding problem is *terrain guarding* — covering a 1.5-dimensional terrain rather than the interior of a polygon — which admits constant-factor approximations [15] that were subsequently improved and extended to weighted and partial-coverage variants [16]. The vertex-guard variant has a long history of approximation algorithms and exact integer-programming approaches: Ghosh [17] gives approximation algorithms with a logarithmic factor for guarding simple polygons, Couto et al. [18] compute exact vertex-guard optima via integer programming, and we treat their published instance library [19] as the source of polygons in our experiments. Classical greedy heuristics [20] place guards to maximize marginal coverage and are reliable on small to mid-sized polygons; local-search refinement of a candidate guard set by swapping or removing vertices is the standard high-quality post-processing step. We use classical greedy purely as a non-learned evaluation reference, not in competition with the reinforcement method on guard count. The supervised targets for the representation probe (Section 4.2) are *not* produced by a policy-independent classical solver: they come from a best-improvement editor (add/remove/swap) initialized at the policy’s own greedy-decoded seed and scored on discretized visibility (Section 5.2). We refer to this policy-seeded refinement as “LS” throughout; it is the same edit family whose *learned* counterpart we analyze in Section 6.4, here run to its best-improvement optimum as an oracle teacher rather than learned.

Reinforcement learning has been applied to the AGP and to closely related coverage problems before, but in settings quite different from ours. Liao et al. [21] discretize the polygon into a grid and train a per-instance tabular Q-learning agent that trades guard count against covered area; Kaba et al. [22] cast the 3D view-planning problem — a sensor-coverage set-cover — as an MDP solved with value-based RL (SARSA, Q-learning, temporal-difference) guided by a geometry-aware score, outperforming the greedy baseline. Both methods consult coverage or visibility while placing guards and learn a fresh policy for each instance. We differ on three axes: the action space is the polygon’s own vertices rather than grid cells or candidate viewpoints; a single amortized policy generalizes to unseen polygons without retraining; and inference is geo-free, with no visibility computation at the moment guards are placed.

Reinforcement learning for combinatorial optimization.. Pointer networks [12] introduced the architecture of attending over input tokens to produce variable-length output index sequences, and were extended to combinatorial optimization with reinforcement learning by Bello et al. [8] using REINFORCE [7] with a learned baseline; Deudon et al. [23] similarly train TSP construction heuristics by policy gradient. Subsequent work introduced attention-only architectures for routing problems [13], policy-optimization variants such as POMO [14] that exploit problem symmetries, and Transformer pointer policies such as Pointerformer [24] aimed at generalizing to large instances — a concern we share, since our evaluation stresses out-of-distribution polygon sizes. We adopt the

more recent preference-optimization (PO) recipe of Pan et al. [5], which trains the policy with a Bradley–Terry [6] ranking loss over pairs of solutions sampled from the policy itself, replacing the variance-heavy REINFORCE objective. We treat PO as a reinforcement-style procedure: the loss is computed from policy rollouts on the environment (the polygon), the gradient direction is determined by the reward (coverage versus guard usage), and no supervised labels for the action sequence are used. A broader methodological survey is provided by Bengio et al. [25].

A recurring feature of this literature is worth making explicit, because AGP departs from it. Routing problems such as the TSP carry a single scalar objective and free feasibility — any permutation is a valid tour — and even constrained variants such as capacitated routing usually enforce feasibility by masking infeasible actions at decode time, which presumes an oracle for the constraint. Vertex-guard AGP has neither property: it is a constrained set-cover problem (Section 2.2), and under geo-free inference the coverage constraint cannot be masked in, so it can only be learned and traded off against guard count — the operating curve our threshold sweep traces. The closest learned analogues are RL methods that approximate Pareto frontiers for bi-objective routing [26, 27]; but those balance two *soft* costs, whereas one of our two axes is a hard feasibility constraint, making PA-Net’s Lagrangian relaxation of a constraint [26] a closer match than a symmetric Pareto front. On the optimization side, Caramanis et al. [28] prove that policy-gradient solution-samplers enjoy benign landscapes for a broad class of problems (Max-/Min-Cut, Max- k -CSP, bipartite matching, TSP), but their guarantee assumes a single cost that is *bilinear* in instance and solution features and imposes no hard feasibility constraint. AGP’s guard-count term is linear and would fit, but its coverage term is the submodular union of visibility regions and its full-coverage requirement is a hard, geo-free-uncertifiable constraint — placing AGP outside their analyzed class and helping explain why a naive policy gradient struggles here (Section 4.1).

Supervised post-processing of learned solvers.. An orthogonal line of work post-processes a base solver’s output with a second model trained by supervised or imitation learning. For combinatorial problems with hard feasibility constraints (such as covering), iterative supervised editors must contend with compounding errors and a train–inference distribution gap; DAgger [29] and its variants partially address the latter. We use a single-shot supervised post-processor (the SETPREDICTOR) not as a competing solver but as a *probe* of the RL-trained encoder’s representation. The framing is methodological: if a small classifier reading frozen RL embeddings can recover feasibility, the RL encoder must already carry the relevant geometric information. Directly cloning search solutions with a recurrent decoder, by contrast, fails on this problem for a structural reason — the teacher-forcing cascade we return to in Section 6.4 — which is why we read the frozen encoder with a single-shot, recurrence-free probe rather than a sequential supervised decoder.

Probing learned representations.. The methodology of freezing a trained network and reading its internal representations with a small supervised classifier originates in the analysis of vision and language models: linear classifier probes diagnose what intermediate layers encode [9], control tasks calibrate how much of a probe’s accuracy reflects the representation rather than the probe’s own capacity [10], and Belinkov [11] surveys the family. We adapt this methodology to neural combinatorial optimization, where it has seen little use: the probed model is a *reinforcement-trained* policy rather than a self-supervised encoder, the probed property is a geometric feasibility signal (guard membership) rather than linguistic structure, and the geo-free constraint fixes what the probe may read at inference. Our no-encoder ablation (Section 6.2) plays the role of the control task: it keeps the probe’s capacity and training recipe constant while removing the representation, so the performance gap is attributable to the encoder.

4. Method

We describe the reinforcement method (Section 4.1) and the single-shot representation probe (Section 4.2); the empirical diagnostic that most motivates the single-shot design — why iterative editors fail to improve the seed — is deferred to the results (Section 6.4).

4.1. The reinforcement method: PO/BT pointer policy

Architecture.. The policy is a compact pointer network with a single-layer, unidirectional LSTM [30] encoder and an attention-based decoder. A polygon P with vertex sequence $V(P) = (x_1, \dots, x_n)$ is consumed coordinate-wise (no visibility input). Running the LSTM over the vertex sequence yields per-vertex hidden states that serve as the embeddings $\{h_i\}_{i=1}^n$, $h_i \in \mathbb{R}^d$, where d is the encoder embedding dimension (its value, and those of all dimensions introduced below, is set in Section 5.2). The decoder attends over the encoder at each step, emits a vertex index or an EOS token, and stops at EOS or when the maximum length is reached. The decoder outputs a sequence $\pi = (\pi_1, \dots, \pi_T)$; the resulting guard set is $S(\pi) = \{\pi_t : \pi_t \neq \text{EOS}\}$, and all metrics (coverage, $|S|/n$, $|S|/\text{OPT}$) are computed on $S(\pi)$. The encoder acquires its geometric representation only as a by-product of training the full policy — the reward gradient is back-propagated from the decoder’s rollouts into the encoder, which has no training signal of its own. This is what dictates the probing methodology: because the only encoder that exists is the one inside the trained policy, we read the representation by *freezing that encoder* and attaching a probe to it, rather than training a fresh encoder-and-classifier from scratch — a standalone model would reveal nothing about what the reinforcement-trained policy learned. The policy’s greedy decode additionally supplies the seed guard set that the representation probe (Section 4.2) later refines.

Why an autoregressive pointer.. Vertex-guard placement is a coverage (set-cover) problem: the value of a guard depends on what the already-chosen guards leave uncovered. Conditioning each pick on the partial solution factorizes the guard-set distribution $p(S \mid P) = \prod_t p(\pi_t \mid \pi_{<t}, P)$ into conditionals that mirror the greedy marginal-coverage rule known to be near-optimal for submodular cover, and it lets the policy decide cardinality endogenously through the EOS token rather than through a global threshold. A model that scored each vertex independently could not represent this conditional structure — which is precisely why the single-shot probe of Section 4.2 consumes the policy’s seed as an input and serves as a representation *probe* rather than a standalone solver. This pre-empts an objection our own results invite: if reading the encoder closes the tail, why keep the decoder at all? Because the encoder owes its representation to the decoder. The encoder has no training signal of its own (its gradient is back-propagated from the decoder’s rollouts), and the probe is not a free-standing classifier — it reads the decoder’s seed as a feature (Eq. (6)), so removing the decoder removes both the representation and the probe’s input. The decomposition we report is encoder-*and*-decoder versus encoder-and-probe, not decoder versus no-decoder. The split is also what buys generalization: pointer policies are known to degrade on instances larger than those seen in training [12, 24], and reading the frozen encoder with a small probe is how we recover feasibility at n far beyond the training range (Section 6.3) without retraining the policy.

Reward and rollouts.. For each polygon P we sample R rollouts from the policy and score each rollout’s guard set $S(\pi)$ by a scalar utility that gates coverage at τ and then prefers smaller guard sets,

$$u(\pi \mid P) = \min(\text{Cov}(S(\pi)), \tau) - \lambda \frac{|S(\pi)|}{n} - \rho \max(0, \tau - \text{Cov}(S(\pi))), \quad (3)$$

where $\tau = 0.99$ is the coverage gate, λ weights guard usage, and ρ penalizes coverage below the gate. Capping the coverage term at τ removes any incentive to over-cover, so once a rollout is feasible the utility is driven purely by cardinality. Coverage during training is computed with a discretized-visibility sampler ($M = 500$ interior points per polygon) as a cheap approximation; CGAL exact visibility is reserved for evaluation.

Preference-optimization loss.. We score a rollout by its *length-normalized* log-likelihood — the mean per-token log-probability under the factorization $p_\theta(\pi \mid P) = \prod_t p_\theta(\pi_t \mid \pi_{<t}, P)$,

$$\bar{\ell}_\theta(\pi \mid P) = \frac{1}{|\pi|} \sum_{t=1}^{|\pi|} \log p_\theta(\pi_t \mid \pi_{<t}, P), \quad (4)$$

which keeps the preference from being biased by guard-set size, since AGP solutions have variable length (cf. Pan et al. [5]). For each pair (π^+, π^-) of rollouts with $u(\pi^+) > u(\pi^-)$ above a coverage gate, we apply the Bradley–Terry log-likelihood

$$\mathcal{L}_{\text{BT}}(\theta) = -\mathbb{E}_{(P, \pi^+, \pi^-)} [\log \sigma(\alpha [\bar{\ell}_\theta(\pi^+ \mid P) - \bar{\ell}_\theta(\pi^- \mid P)])], \quad (5)$$

Training objective	Mean cov	Mean $ S /\text{OPT}$
REINFORCE, learned-critic baseline	0.904	3.49
REINFORCE, greedy rollout baseline	1.000	7.00
PO / Bradley–Terry (ours)	0.978	1.30

Table 1: Training-objective comparison on the `dev` split (greedy decode). Coverage is geometric polygon coverage; $|S|/\text{OPT}$ is the mean approximation ratio. The REINFORCE rows are 30-epoch runs evaluated under an earlier version of the evaluation protocol; they are reported as evidence for the choice of training objective, not as tuned, protocol-matched competitors. The PO/BT row is the policy used throughout the paper.

where $\alpha > 0$ is a temperature scaling the preference margin. The gradient direction is driven purely by which of the two rollouts achieved a higher reward; no supervised target sequence is used.

We use PO rather than REINFORCE deliberately. Pan et al. [5] report that binary preferences over pairs of rollouts retain a useful gradient direction even when rewards saturate — the regime a value-baseline REINFORCE [8] struggles with on dense-reward problems — and on AGP we observe exactly this failure: a value-critic REINFORCE policy over-guards by roughly $3.5\times$ the optimum and a greedy-baseline variant by about $7\times$, while PO/BT reaches comparable coverage at $1.3\times$ (Table 1). The cause is structural and is exactly the constrained nature of the problem (Section 2.2) showing up in the optimization: under the coverage-gated reward of Eq. (3), once a rollout meets the coverage constraint the reward flattens and the policy-gradient signal on the guard-cost term vanishes. This gate-induced plateau is a reward-saturation effect of the hard coverage constraint — distinct from the near-optimum gradient-variance decay analyzed for unconstrained, bilinear-cost samplers [28], and not covered by those benign-landscape guarantees. Bradley–Terry preferences between rollouts, by contrast, stay informative even when both saturate, preserving a usable gradient on the cardinality axis. At the end of training we are left with a single deterministic policy that maps a polygon’s coordinates to a guard sequence in one forward pass; this is the artifact whose properties the rest of the paper investigates, and from this point on its encoder is frozen and treated as a learned feature extractor. Figure 1 reports the held-out training dynamics across four reconstructed checkpoints and shows that coverage and cost move jointly over training rather than the optimization collapsing onto a single axis (all coverage and no cost control, or the reverse).

4.2. Probing the representation: a single-shot per-vertex classifier

The policy is strong on cardinality but leaves a feasibility tail (Section 6.1), and a learned *iterative* editor over its seed does not close that tail in any coverage-preserving regime (Section 6.4): error accumulation, stop-time miscalibration, and train/inference drift make a rollout-based post-processor unreliable here, contrary to fine-tuning arguments of the policy in [5]. We therefore probe what the policy has internalized with a *single-shot, rollout-free* classifier — conditioned only on geo-free inputs (the frozen encoder embedding, the

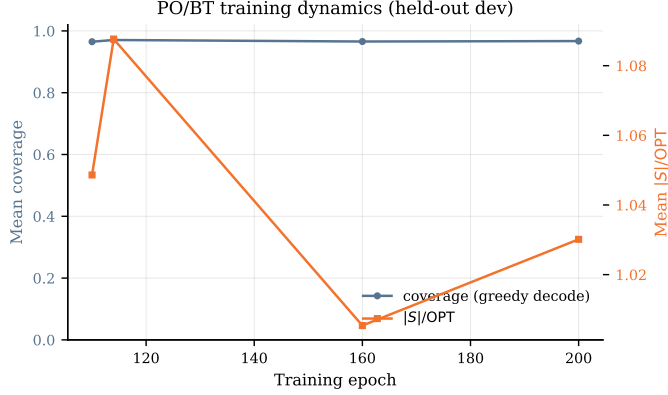


Figure 1: PO/BT training dynamics on the held-out **dev** split, reconstructed post-hoc from the four saved checkpoints — not a continuous online log; the early phase of training is not covered (see Section 8). Mean per-polygon coverage (left axis) and the approximation ratio $|S|/\text{OPT}$ (right axis) of the greedy-decoded policy. Over the late phase shown, the coverage–cost trade-off improves jointly rather than collapsing onto a single axis.

vertex coordinates, and the policy’s seed indicator) and never on a visibility object at inference — that recovers the feasibility decision the decoder failed to make. Having no rollout and no stop decision, a single pass has nothing to drift and nothing to miscalibrate. We call it the SETPREDICTOR and present it primarily as a representation diagnostic; the threshold sweep in Section 6 makes the diagnostic concrete, but the operating-curve framing is a side-effect of the measurement, not a deployment recipe.

For polygon P with vertex sequence $V(P) = (x_1, \dots, x_n)$, the input feature for vertex i is

$$z_i = [h_i; x_i; \mathbf{1}\{i \in \pi_{\text{seed}}\}] \in \mathbb{R}^{d+3}, \quad (6)$$

that is, the *frozen* d -dimensional encoder embedding h_i from the PO/BT policy, the 2D coordinates x_i , and a single binary indicator for whether vertex i is selected by the policy’s greedy-decoded seed π_{seed} . No visibility matrix, no CGAL output, and no area feature enters the classifier at inference time. The geo-free constraint is preserved.

The architecture is a small Transformer encoder over the n vertex tokens with a sigmoid output head per token (sizes in Section 5.2). The features z_i are linearly projected to width d and passed through L pre-norm self-attention blocks — each with H -head self-attention at width d , a feed-forward sublayer of expansion factor f , and residual connections — followed by a final layer normalization; we write v_i for the resulting per-vertex hidden state. This v_i is the SETPREDICTOR’s *own* encoding of vertex i , and is distinct from the frozen policy embedding h_i , which enters only as part of the input feature z_i in Eq. (6). From the $\{v_i\}_{i=1}^n$ we build two context vectors by mean-pooling: a *seed* context $S_{\text{pool}} = \frac{1}{|\pi_{\text{seed}}|} \sum_{i \in \pi_{\text{seed}}} v_i$, averaged over the vertices the policy placed

in its seed, and a *global* context $G_{\text{pool}} = \frac{1}{n} \sum_{i=1}^n v_i$, averaged over all vertices of the polygon. Both are single vectors shared across vertices. Each per-vertex prediction concatenates the vertex’s own v_i with these two context vectors and passes the result through a 2-layer MLP head ($3d \rightarrow d \rightarrow 1$):

$$\hat{p}_i = \sigma(\text{MLP}([v_i \parallel S_{\text{pool}} \parallel G_{\text{pool}}])), \quad i = 1, \dots, n. \quad (7)$$

The output $\hat{p}_i \in [0, 1]$ is interpreted as the probability that vertex i should be in the final guard set. Thresholding at $t \in (0, 1)$ yields the guard set

$$S(t) = \{i \in \{1, \dots, n\} : \hat{p}_i \geq t\}. \quad (8)$$

Objective.. The probe is trained by supervised learning to predict membership in an LS-improved target set $\pi_{\text{LS}}^*(P)$ — the policy’s seed refined by local search, constructed as described in Section 5.2 — so it is not itself a reinforcement learner. With $y_i = \mathbf{1}\{i \in \pi_{\text{LS}}^*(P)\}$ and a positive-class weight w_+ that balances the empirical guard/non-guard ratio, the per-vertex weighted binary cross-entropy is

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{P \in \mathcal{B}} \frac{1}{|V(P)|} \sum_{i=1}^{|V(P)|} [w_+ y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]. \quad (9)$$

The logic is direct: if a classifier reading only the frozen embeddings h_i (with x_i and the seed indicator) can close the feasibility tail, the encoder must already carry the geometry that decision needs.

Inference and the threshold knob.. At inference we run the policy encoder once on the coordinates, greedy-decode the seed π_{seed} , form the features z_i of Eq. (6), run one SETPREDICTOR forward pass to obtain $\{\hat{p}_i\}_{i=1}^n$, and threshold to return $S(t)$ (Eq. (8)). No CGAL call, visibility matrix, or oracle appears on this path. The threshold t is the single knob trading coverage for guard count, swept over $t \in \{0.20, 0.25, 0.30\}$ in Section 6.2. Re-running the probe with its own output as the updated seed indicator changes nothing — it is empirically a fixed point after the first pass (Section 6.4) — so we use a single pass throughout.

5. Experiments

5.1. Dataset and partition

We draw polygons primarily from the *Art Gallery Problem with Vertex Guards* (AGPVG) instance library of Couto, de Rezende, and de Souza [19, 18], which collects several polygon families — random orthogonal (including large-area **fat** and small-area **min** variants), random simple (**randsimple**), random von Koch (**randvon**), complete von Koch (**vonkoch**), and boundary-simple (**simple**) — over vertex counts n ranging from 8 to 2,500. The library distributes vertex-guard optimal solutions for its instances, which we use directly as OPT.

Split	# polygons	n range	Use
train	9,791	8–150	Pointer + SetPredictor training
dev_tune	857	8–150	SetPredictor checkpoint selection
dev_test	367	8–150	Held-out evaluation (in-distribution)
test	2,107	12–1,000	Out-of-distribution evaluation
large	285	600–2,250	Extreme-OOD evaluation

Table 2: Dataset partition. The **dev** split is partitioned 70/30 into **dev_tune** (checkpoint selection) and **dev_test** (held-out evaluation) by seeded random shuffle (seed 1234). The **test** split contains polygons whose vertex count exceeds the training range and is used as the out-of-distribution evaluation. The **large** split is used for an extreme-OOD evaluation (Table 7).

We supplement the library with 313 additional random simple polygons (**random** class, $n \in \{20, 40, \dots, 600\}$, 30 instances per size) to increase training density for random simple polygons in the in-distribution range; these share the same rational-coordinate format, and their vertex-guard optima were computed by solving the vertex-guard set-cover integer program. The splits we work with are listed in Table 2.¹

5.2. Training setup

The PO/BT policy is trained once and frozen — the existing **lstm_bt** checkpoint, with all PO coverage rewards computed from discrete visibility sampling ($M = 500$) — and a single seed (1234). The SETPREDICTOR probe is trained on **train** only. *Targets*: for each training polygon we obtain $\pi_{LS}^*(P)$ by running local search on the policy’s seed under a coverage gate $\tau = 0.99$ against a discrete-visibility reference; CGAL exact visibility is used only at evaluation. *Sizes*: the model dimensions of Section 4.2 take the following values.

working width d	128
self-attention blocks L	3
attention heads H	8
FFN expansion factor f	2
policy parameters	$\approx 364k$
SETPREDICTOR parameters	464,001

Optimization: weighted BCE (Eq. (9)) with positive-class weight $w_+ \approx 5.66$ (the empirical guard/non-guard ratio on **train**), 60 epochs, batch size 32, learning rate 3×10^{-4} , AdamW [31]. Both the headline probe and the no-encoder ablation are four independent runs with seeds $\{1234, 11, 22, 33\}$ (reported as mean $\pm$ std); all figures display seed 1234. Checkpoints are selected on **dev_tune**; **dev_test**,

¹The 70/30 **dev** partition uses a seeded random shuffle (seed 1234) rather than an alphabetical-name split: the latter had concentrated **fat-*** polygons in **dev_tune** and **randsimple-*** in **dev_test**, producing misleading optimism that the random shuffle removes.

`test`, and `large` are touched only after training and threshold selection are complete.

5.3. Evaluation protocol and metrics

For each polygon we greedy-decode the policy’s EOS-truncated seed and evaluate it with CGAL exact polygon visibility [32]; separately, we run one SETPREDICTOR pass, threshold at t to obtain $S(t)$, and evaluate $S(t)$ with CGAL. Let N be the partition size. We report mean coverage $\frac{1}{N} \sum \text{Cov}(S)$, the normalized guard count $|S|/n$, and the approximation ratio $|S|/\text{OPT}$ against the vertex-guard optimum. Because the mean hides the tail, we also report the per-polygon counts $\#\{\text{Cov}(S) \geq c\}$ for $c \in \{0.95, 0.99, 0.999, 1\}$ and the worst case $\min_P \text{Cov}(S)$; we call $\text{Cov}(S) \geq 0.95$ the *feasibility* threshold and report the feasibility rate $\#\{\text{Cov}(S) \geq 0.95\}/N$ with Wilson 95% confidence intervals. Where we compare the policy’s and the probe’s feasibility rates on the same polygons, we test the paired per-polygon outcomes with an exact McNemar test. The reporting threshold c is distinct from the training-time gate $\tau = 0.99$ of Eq. (3). As noted in Section 5.1, OPT values are taken from the vertex-guard optimal solutions distributed with the AGPVG library (for library instances) or computed via ILP (for the `random` class). For 79 of the `large` polygons (all with $n \geq 800$) no optimum is available — most are library instances whose distributed solutions do not cover them (they are open at the source), and for the remainder our ILP did not terminate within budget. Restricting $|S|/\text{OPT}$ to the 206 solved instances would bias the ratio toward the easier 600–700-vertex polygons, so on `large` we report coverage and the normalized guard count $|S|/n$ over all 285 polygons but no approximation ratio.

6. Results

The results follow the hypothesis: the policy places guards but leaves a tail (Section 6.1), reading the frozen encoder closes it (Section 6.2), the effect generalizes out of distribution (Section 6.3), and a single pass suffices because the probe is a fixed point under iteration (Section 6.4).

6.1. The policy places guards

The geo-free policy on its own (Table 3, *seed* row) places guards at near-classical cardinality — its mean $|S|/n$ (0.166) is close to classical greedy (0.152) and the oracle editor it is later refined toward (0.137), at $|S|/\text{OPT} \approx 1.09$ — and clears the feasibility line on the large majority of `dev_test` polygons. This confirms that vertex-guard placement is learnable, to a degree, by a method that never reads an explicit visibility object at inference. The residual is the honest scope: 71 of the 367 polygons fall below the feasibility line under the policy alone (the distribution is Section 6.2), so the method gives no feasibility guarantee on its own. Figure 2 shows the seed, the seed plus probe at $t = 0.20$, and the optimum side by side for an in-distribution and an out-of-distribution polygon.

Method	Mean cov	$\#\{\text{Cov} \geq .95\}/N$	Mean $ S /n$	Mean $ S /\text{OPT}$
<i>Classical baseline (non-learned)</i>				
Greedy	0.9994	367/367	0.1523	1.0086
<i>Learned policy / probe</i>				
Pretrained pointer (seed)	0.9689	296/367 (0.763, 0.844)	0.1664	1.0878
SetPredictor full ($t = 0.20$)	0.9984 ± 0.0011	366.2 ± 1.3 /367	0.3890 ± 0.0606	2.5593 ± 0.4043
SetPredictor, no-encoder ($t = 0.20$)	0.9972 ± 0.0013	366.8 ± 0.4 /367	0.4783 ± 0.0503	3.1087 ± 0.3348
<i>Oracle editor (SETPREDICTOR target, policy-seeded)</i>				
LS on policy seed	0.9948	367/367	0.1367	0.8840 [†]

Table 3: Held-out evaluation on `dev.test` (367 polygons). Classical greedy is the only non-learned anchor (guard set from [2]-style farthest-coverage). The *seed* row is the PO/BT-trained policy decoded greedily. The bottom *LS on policy seed* row is *not* a classical baseline: it is the policy’s greedy seed refined by a best-improvement editor (add/remove/swap) scored on discretized visibility (Section 5.2) — the supervised target the SETPREDICTOR is trained to predict, listed here as the reference the probe imitates. SETPREDICTOR *full* is the headline representation probe; numbers are mean $\pm$ std across four probe seeds {1234, 11, 22, 33} at fixed threshold $t = 0.20$. SETPREDICTOR, *no-encoder* has the same architecture and parameter count with the frozen encoder masked to zeros, aggregated over the same four seeds (mean $\pm$ std; Section 6.2). The 0.95 gate is saturated for both probes ($\geq 366/367$), so it does *not* expose the encoder’s effect; that separation appears at the stricter 0.99/0.999 gates of Table 4, and here in the $\approx 20\%$ higher guard cost the no-encoder probe pays for matched coverage ($|S|/\text{OPT}$ 3.11 versus 2.56). Wilson 95% CI is shown for the policy seed only, where the feasibility proportion is informative; rows that reach 367/367 omit the degenerate CI.

[†]The oracle editor reports $|S|/\text{OPT} < 1$ not because OPT is loose — it is the verified library/ILP optimum (Section 5) — but because the editor halts at its disc-vis coverage gate ($\tau = 0.99$) and slightly undercovers (mean exact coverage 0.995, below full coverage on 95% of polygons), so it spends fewer guards than the *full-coverage* optimum. This is an artifact of the cardinality-trading reward; $|S|/n$ is the cardinality metric without this caveat.

6.2. Reading the encoder closes the tail

The policy’s mean coverage hides a bimodal distribution (Table 4): it reaches full coverage on only a handful of polygons and trails a long left tail down to a worst polygon at 0.830, while the SETPREDICTOR at $t = 0.20$ eliminates that tail and lifts the bulk above 0.99. On the displayed seed the probe clears all 71 of the policy’s sub-feasibility polygons and introduces no new failures ($71 \rightarrow 0$ below 0.95; paired exact McNemar test, $p \approx 8 \times 10^{-22}$); the same paired test is applied at OOD scale in Section 6.3.

Is closing that tail the encoder’s doing, or could coordinates alone suffice? We isolate the encoder’s contribution by retraining the probe with the frozen embedding h_i masked to zero on every forward pass — architecture, parameter count, optimizer, and training schedule identical (Section 5.2). The 0.95 feasibility gate is the wrong place to look: both probes saturate there (367/367, Table 3), and the headline mean coverage is likewise nearly identical. The encoder’s contribution lives in the tail. At the stricter 0.99 gate of Table 4, on the displayed seed the full probe leaves 4 polygons below against 57 for the no-encoder ablation — a fourteen-fold difference — and lifts the worst polygon from 0.952 to 0.977 (the four-seed mean count below 0.99 is ≈ 11 for the full probe). The gap becomes qualitative out of distribution (Section 6.3). This is the direct measurement behind C1 — the frozen encoder carries geometric structure that raw coordinates do not reproduce, even with the decoder’s full

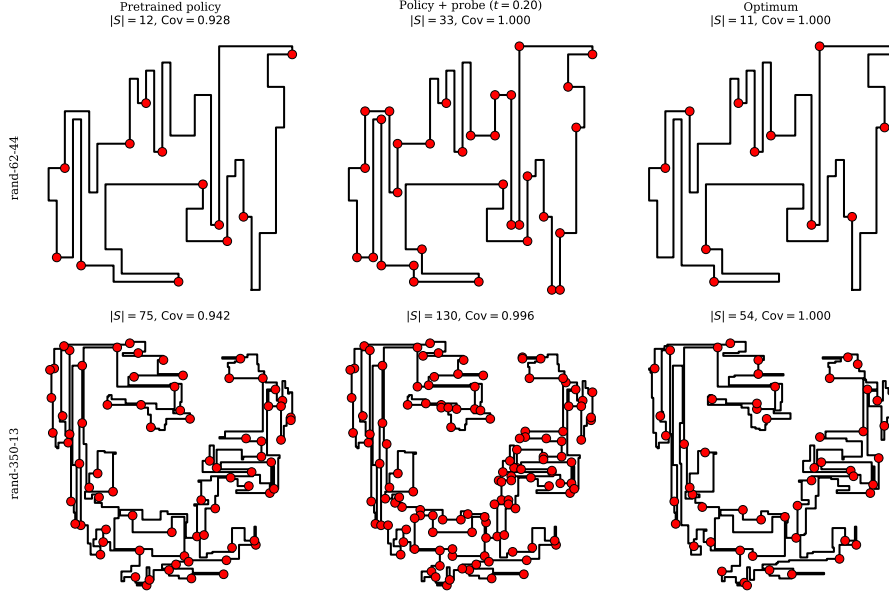


Figure 2: Worked examples: two polygons drawn with the policy’s seed (left column), the policy plus the SETPREDICTOR probe at $t = 0.20$ (middle column), and the optimum (right column). Vertices selected as guards are marked. Row 1 is an in-distribution polygon where the policy already covers most of the region. Row 2 is an out-of-distribution polygon ($n = 350$, about $1.8\times$ the largest training polygon) where the policy’s seed falls below the feasibility line ($\text{Cov}(\cdot) = 0.942$) and the probe lifts it back above ($\text{Cov}(\cdot) = 0.996$); its guard cost here ($|S|/\text{OPT} \approx 2.4$) is moderate and within the method’s typical range on this split rather than a worst case (cf. Table 6).

parameter budget.

One objection remains: the SETPREDICTOR is itself a small Transformer, so perhaps *it* computes the geometry and the encoder merely passes coordinates through. To rule this out we strip the probe of all capacity and ask how linearly separable guards are in the frozen embedding alone. A plain L_2 -regularized logistic regression — no hidden layer, no attention — fit directly on the frozen per-vertex embeddings against the LS-derived guard-membership label reaches $\text{ROC-AUC} = 0.842 \pm 0.024$ (5-fold polygon-grouped cross-validation, 200 polygons, 12,268 vertices, mean $\pm$ std across folds; $\text{PR-AUC} = 0.580 \pm 0.048$ against a positive-class prior of 0.16). ROC-AUC here is just how reliably such a linear rule separates guard from non-guard vertices (0.5 is chance, 1.0 perfect). A linear readout cannot be credited with computing geometry itself [9, 10], so guard-relevant structure must already be linearly accessible in the embedding space — the encoder, not the probe, carries it. Figure 5(b) visualizes the same separation in a 2D projection.

Closing the tail has a price, with the threshold t as the knob: the approximation ratio rises as t falls (Table 5), reaching $\approx 2.6\times$ optimum at the headline

Method	$\#\{\text{Cov} = 1\}$	$\#\{\text{Cov} \geq .999\}$	$\#\{\text{Cov} \geq .99\}$	$\#\{\text{Cov} \geq .95\}$	min Cov
Pretrained pointer (seed)	3	15	99	296/367	0.830
SetPredictor $t = 0.20$ (full)	243	315	363	367/367	0.977
SetPredictor $t = 0.20$, no-encoder	122	166	310	367/367	0.952
SetPredictor $t = 0.25$ (full)	195	283	361	367/367	0.962
SetPredictor $t = 0.30$ (full)	152	221	355	367/367	0.952

Table 4: Per-polygon coverage distribution on **dev_test** (367 polygons). Counts show the number of polygons reaching each coverage threshold; $\min \text{Cov}(S)$ is the worst polygon. The pretrained policy has a long left tail; the SETPREDICTOR moves the entire distribution to the right. This is the table that exposes the encoder’s contribution: at the 0.95 gate both probes saturate (367/367), so the separation shows only at the stricter 0.99/0.999 gates and in $\min \text{Cov}(S)$. At $t = 0.20$ the full probe leaves 4 polygons below 0.99 against 57 for the no-encoder ablation (the matched-capacity control of Section 6.2), and lifts the worst polygon from 0.952 to 0.977. The *full* rows and the no-encoder row are the displayed seed (1234); Table 3 reports the four-seed mean $\pm$ std for the full probe at $t = 0.20$.

Threshold t	Mean Cov	Mean $ S /n$	Mean $ S /\text{OPT}$	$\#\{\text{Cov} \geq .95\}/N$
0.20	0.9993	0.4376	2.9007	367/367
0.25	0.9988	0.3802	2.5091	367/367
0.30	0.9979	0.3352	2.2030	367/367

Table 5: Cost of feasibility on **dev_test** (367 polygons), displayed seed (1234). The four-seed mean $|S|/\text{OPT}$ is 2.02 ± 0.21 at $t = 0.30$ and 2.56 ± 0.40 at $t = 0.20$.

threshold against ≈ 1.1 for the policy alone. Figure 3 gives the full picture in and out of distribution: the coverage CDF sits to the right of the policy at every threshold, while the $|S|/n$ and $|S|/\text{OPT}$ box plots show the matching cost. We report the trade-off rather than fix one operating point.

6.3. Out-of-distribution generalization

The strongest test of the representation is the OOD **test** split, whose polygons reach $n = 1000$ — about five times the largest training polygon. Table 6 reports it. The policy alone drops to mean coverage 0.957, with 310/2107 polygons below the feasibility line (85% feasible). The probe at $t = 0.20$ (four-seed mean) cuts that to 17.5 ± 12.5 (99% feasible) at mean coverage 0.995 — roughly an order-of-magnitude reduction in failures. The improvement is significant for every probe seed individually (per-seed failure counts 3, 8, 25, 34; exact McNemar test on the paired per-polygon outcomes, $p < 10^{-75}$ in all four cases), and the per-seed Wilson 95% intervals for the probe’s feasibility rate all lie above 0.977. The no-encoder ablation leaves 44 ± 23 below feasibility (four-seed mean) at mean coverage 0.985 — about $2.5\times$ the full probe’s failures (Table 6), so out of distribution the encoder’s contribution is qualitative, not the saturated tie it is in distribution. This is the evidence behind C2: a small classifier over the frozen embeddings recovers feasibility on polygons far larger than any the encoder saw in training. The reduction is robust across seeds; what varies is where each seed’s failures concentrate along the size axis, so at-scale generalization is partially seed-dependent, with mean coverage staying above 0.97 across the full

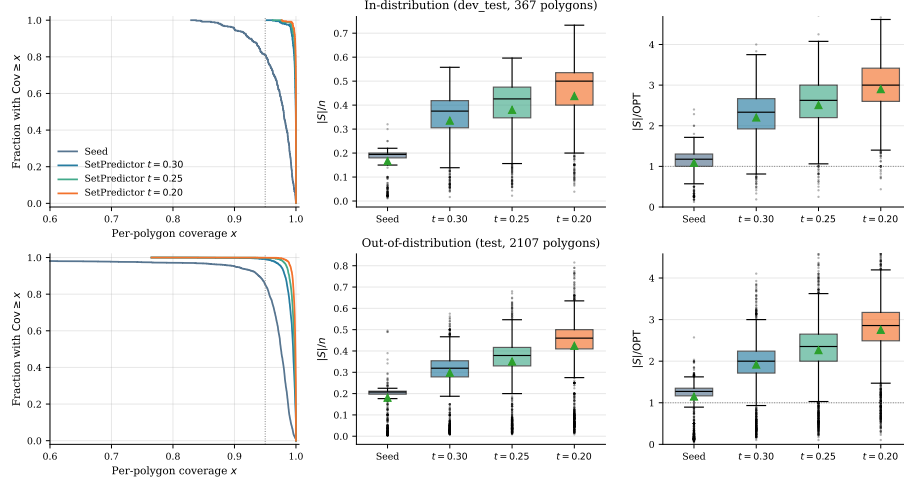


Figure 3: Distributional view of method performance, in-distribution `dev_test` (top, 367 polygons) and out-of-distribution `test` (bottom, 2107 polygons). *Left*: per-polygon coverage as a complementary stair-step CDF (fraction of polygons with coverage at least x); the dashed line marks the 0.95 feasibility gate. *Center, right*: box plots of the normalized guard count $|S|/n$ and the approximation ratio $|S|/OPT$ for the policy seed and the three probe thresholds. Boxes show the displayed seed (1234); the four-seed mean $\pm$ std is in Table 3.

Method	Mean cov	$\#\{Cov \geq .95\}/N$	Mean $ S /n$	Mean $ S /OPT$
Pretrained pointer (seed)	0.9568	1797/2107 (0.837, 0.867)	0.1807	1.1494
SetPredictor full ($t = 0.20$)	0.9946 $\pm$ 0.0020	2089.5 $\pm$ 12.5/2107	0.3617 $\pm$ 0.0447	2.3367 $\pm$ 0.2948
SetPredictor, no-encoder ($t = 0.20$)	0.9850 $\pm$ 0.0055	2063.0 $\pm$ 23.0/2107	0.3679 $\pm$ 0.0549	2.3680 $\pm$ 0.3639

Table 6: Out-of-distribution evaluation on `test` (2107 polygons, n up to 1000). Rows as in Table 3: the policy seed, the full probe, and the no-encoder ablation, both probes the four-seed mean $\pm$ std at $t = 0.20$.

n range. Figure 4 disaggregates coverage by polygon size, and the bottom row of Figure 3 shows the matching distributions.

Extreme out-of-distribution.. On the `large` split (n from 600 to 2250, up to about eleven times the largest training polygon) the worst case, not the mean, is the metric (Table 7). The pretrained seed leaves its worst polygon at coverage 0.038 with 75/285 below feasibility, and the no-encoder ablation barely moves it: its worst-polygon coverage averages only 0.09 ± 0.09 across the four seeds (three of the four still pinned at 0.038), so it trails the full probe on the mean (0.930 versus 0.963) and collapses on the tail. The full probe lifts the worst polygon for every seed: the four seeds the worst-polygon coverage at $t = 0.20$ averages 0.61 ± 0.29 (against the no-encoder’s 0.09 ± 0.09), and on the displayed seed reaches 0.951 with nothing left below feasibility. The seed dependence is genuine — the four-seed feasibility count is 253.5 ± 19.9 of 285, with the per-seed count below feasibility ranging from 0 (the displayed seed) to 55. The probe spends about twice the guards of the local-search reference it

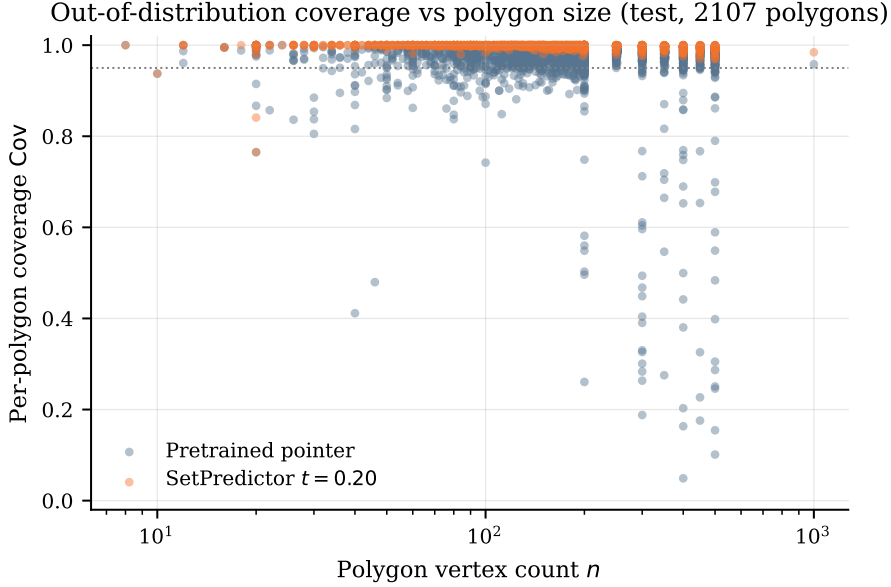


Figure 4: Per-polygon coverage as a function of vertex count n on the OOD test split ($n \in [12, 1000]$). The pretrained policy’s coverage degrades systematically as n grows; the SETPREDICTOR probe at $t = 0.20$ stays near the 0.95 feasibility line across the entire size range. Shown for the displayed seed (1234); across the four probe seeds the worst-polygon coverage at $t = 0.20$ ranges from 0.576 to 0.884 (Section 6.3).

imitates ($|S|/n \approx 0.30$ versus 0.15) and in return exceeds that reference’s own feasibility rate on this split (61%).

6.4. Iteration: editors drift, the probe is a fixed point

Before settling on the single-shot probe we tried to improve the seed with a learned *iterative* editor applying ADD/REMOVE/STOP edits. Three architectures of increasing capacity — including one with topology features and an auxiliary visibility-prediction loss, and one with DAGger refresh [29] — failed to improve over the seed in any coverage-preserving regime. The best variant cut $|S|$ by about 24% but lost coverage, reaching only a 0.27 *recovery rate* (the fraction of its edits that match or beat the local-search reference on both coverage and guard count) — well below replacement quality. Three structural reasons recur: errors accumulate because the editor sees clean trajectories in training but its own predictions at inference; a single $\{\text{ADD}, \text{REMOVE}, \text{STOP}\} \times \{1, \dots, n\}$ head couples selection with termination, miscalibrating STOP; and the train/inference state distributions diverge in ways DAGger refresh did not close.

The same teacher-forcing cascade also defeats a *supervised* pointer network — which we trained directly on AGPVG solutions before adopting the reinforcement recipe. Cloning a guard sequence is ill-posed for a second reason beyond the cascade: a polygon has many optimal and near-optimal guard sets (the

Method	Mean Cov	min Cov	$\#\{\text{Cov} \geq .95\}/N$	Mean $ S /n$
Pretrained pointer (seed)	0.8678	0.038	210/285	0.1849
SetPredictor full ($t = 0.20$)	0.9630 ± 0.0209	0.612 ± 0.286	$253.5 \pm 19.9/285$	0.2951 ± 0.0579
SetPredictor, no-encoder ($t = 0.20$)	0.9295 ± 0.0196	0.091 ± 0.092	$251.0 \pm 13.8/285$	0.3053 ± 0.0477

Table 7: Extreme out-of-distribution evaluation on the **large** split (285 polygons, n from 600 to 2250), coverage scored by exact CGAL at $t = 0.20$. The approximation ratio is omitted because the ILP optimum is unavailable for 79 of these polygons (all with $n \geq 800$; Section 5.3); the reported columns are over all 285. Both the full-probe and no-encoder rows are the four-seed mean $\pm$ std, and their minCov is the mean $\pm$ std of the four per-seed worst-polygon coverages.

constraint $\text{Cov}(S) = 1$ in Eq. (2) fixes the cardinality, not the membership), so any single target sequence is an arbitrary choice among equivalent solutions, and an autoregressive loss penalizes the model for proposing a different but equally valid one. Forced onto sequences it would not itself generate, the recurrent decoder then drifts into hidden states unseen at training and collapses (mean coverage ≈ 0.50 at roughly $3\times$ the optimum) — the brittleness of supervised cloning anticipated in Section 1. Preference optimization sidesteps both problems: it ranks the policy’s *own* rollouts by reward rather than matching a designated target, so non-unique optima are not in conflict. The single-shot probe likewise has none of these failure modes — no rollout, no stop token, and a per-vertex membership label rather than an ordered sequence, trained and used in the same regime.

A clarification this invites: the probe’s supervised targets π_{LS}^* are themselves the output of this same edit family run to its best-improvement optimum — an *oracle* editor seeded by the policy. So the contrast is sharper than “editors fail, the probe works”: the *learned* editor fails to reproduce the oracle’s edits autoregressively, whereas a single-shot probe trained on the oracle’s endpoints recovers them in one pass. That the probe succeeds is therefore not, by itself, evidence about the encoder — it could merely be distilling the oracle. What attributes the success to the *representation* is the pair of controls in Section 6.2: holding the target fixed, masking the frozen embedding to zeros (the no-encoder ablation) collapses the tail, and a zero-capacity linear probe over the same embedding already separates guards (ROC-AUC 0.842). The targets locate *what* to predict; the encoder is what makes it predictable from coordinates alone.

Re-running the probe with its own output as the updated seed indicator does not change the prediction in any meaningful way: at the displayed $t = 0.65$ the changes in mean coverage, $|S|/n$, and $|S|/\text{OPT}$ across $K \in \{1, 2, 3, 5\}$ all stay within a few parts in a thousand (Table 8). Figure 5(a) shows the same picture for all six thresholds: the only visible movement is a single settle between $K = 1$ and $K = 2$ at the two highest thresholds (at most ≈ 0.011 , at $t = 0.8$), beyond which no curve moves by more than 2.4×10^{-3} . This is the fixed-point property (C4): with the seed indicator already among its inputs, a single pass suffices.

Inference passes K	Mean Cov	Mean $ S /n$	Mean $ S /\text{OPT}$
$K = 1$	0.9723	0.1968	1.2328
$K = 2$	0.9712	0.1966	1.2302
$K = 3$	0.9719	0.1966	1.2295
$K = 5$	0.9719	0.1966	1.2292

Table 8: Inference passes $K \in \{1, 2, 3, 5\}$ at fixed threshold $t = 0.65$, full **dev** split (1224 polygons). Iterating the SETPREDICTOR returns essentially the same prediction: across K the mean coverage changes by at most 1.1×10^{-3} and the cardinality metrics by less than 4×10^{-3} . We sweep $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$ rather than the headline range, the regime near the decision boundary where the fixed-point property is most stringently tested; Figure 5(a) shows the across- K behavior for all six thresholds.

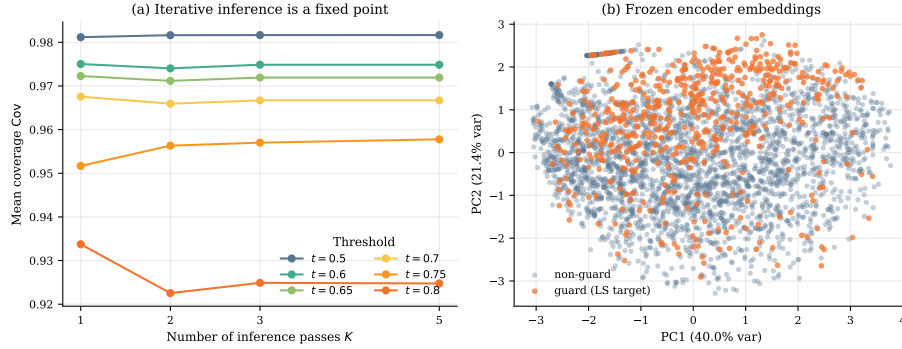


Figure 5: Two views of the probe’s mechanism. (a) Iterative inference is a fixed point: mean coverage as a function of inference passes $K \in \{1, 2, 3, 5\}$ for six thresholds $t \in \{0.5, 0.6, 0.65, 0.7, 0.75, 0.8\}$, on the full **dev** split. The largest one-step settle anywhere is ≈ 0.011 ($t = 0.8$, $K = 1 \rightarrow 2$); beyond $K = 2$ no curve moves by more than 2.4×10^{-3} , and at the tabulated $t = 0.65$ the total across- K variation is $\approx 1.1 \times 10^{-3}$. (b) Two-dimensional projection of the frozen encoder’s per-vertex embeddings, colored by guard-membership label (LS-derived target), pooled across held-out polygons. The separation visible here is corroborated quantitatively by a linear probe over the same d -dimensional embeddings (ROC-AUC 0.842 ± 0.024 , Section 6.2); the 2D scatter is the visualization, the AUC is the measurement.

7. Discussion

The single most informative measurement in this paper is what happens when we read the encoder rather than the decoder. Reading only frozen embeddings and the policy’s seed indicator — no visibility matrix, no CGAL output, no per-vertex area feature — the SETPREDICTOR closes the in-distribution feasibility tail on the displayed seed (Table 3) and compresses the out-of-distribution one by roughly an order of magnitude across four probe seeds (Section 6.3). Two readings are consistent with this. Either the encoder learned enough geometric structure that a small classifier over its output recovers feasibility on its own, or the decoder’s EOS calibration was the bottleneck and the policy had already identified the right vertices but stopped too early. Both point to the same conclusion: the reinforcement method internalized a representation that holds

more of the relevant geometry than its decoder expressed. Two measurements pin this down (Section 6.2). The no-encoder ablation — masking the embedding to zeros with everything else held constant — leaves a probe that cannot match the full probe’s coverage at matched cardinality on either split, separating most clearly at the 0.99 gate of Table 4. And the linear probe (ROC-AUC 0.842 ± 0.024) shows the guard-relevant structure is linearly accessible in the frozen embedding, with Figure 5(b) as the matching visualization.

Recovering feasibility has a price, and we report it rather than disguise it. The probe’s guard sets grow to roughly two to three times the optimum as the threshold is lowered (Section 6.2), and the $|S|/\text{OPT}$ box plots of Figure 3 show that the whole distribution widens, not just its mean. A deployment would pick one operating point on this curve; we leave it as a knob because the right point depends on how costly an extra guard is relative to an uncovered region.

We are not claiming a better AGP solver. Classical greedy reaches mean coverage 0.999 at $|S|/n = 0.152$ and $|S|/\text{OPT} = 1.009$ on `dev_test` (Table 3); the probe at $t = 0.20$ reaches mean coverage ≥ 0.998 at $|S|/\text{OPT} \approx 2.6$ across four seeds. The classical anchors win on cardinality at preserved coverage, and the policy and probe are not built to contest that. What we document is what is learnable by reinforcement when the model is denied geometric input at the moment of placing guards, and what part of that learning lives in the representation rather than the decoder. The contrast between the failed iterative editors (Section 6.4) and the single-shot probe is the cleanest expression of the point: removing the stop time and the rollout is exactly what lets the encoder’s information surface.

8. Limitations

1. **Scope of the “no geometric knowledge” claim.** The geo-free constraint is on inference: the policy and the probe see no visibility object at test time. *Training* does see visibility: the PO/BT reward is computed from disc-vis coverage, and the probe’s supervised targets are generated by local search using disc-vis as a coverage gate. “No explicit geometric knowledge” must be read as “no geometric oracle at the moment of placing guards.”
2. **The probe is supervised, not reinforcement.** The SETPREDICTOR is trained by BCE against local-search-derived labels. It is not itself a reinforcement learner. We position it as a representation probe of the RL-trained encoder, not as an RL contribution.
3. **Held-out partition size.** `dev_test` has 367 polygons. Wilson 95% CIs on the coverage-feasibility proportion are reported, but the absolute count is modest. The OOD evaluation on `test` (2107 polygons) provides a much larger second held-out evidence base.
4. **Policy selection and seeds.** The PO/BT policy is the best of several reinforcement-pretraining runs, selected by training-set performance. Because selection used only the `train` split, `dev_test` and `test` play no role

in it and the held-out evaluations are not optimistically biased by the choice; selecting on training reward rather than a held-out criterion is a weaker rule that, if anything, risks favoring an over-fit run, so we make no claim the selected policy is optimal. We do not average over policy seeds; seed-variance is characterized separately for the probe over four seeds $\{1234, 11, 22, 33\}$ (Table 3, Section 6.3).

5. **LSTM, not Transformer, policy.** We trialed a Transformer encoder-decoder policy of roughly $6\times$ the parameters; it reached a comparable coverage-cost operating point (greedy coverage ≈ 0.97 at $|S|/\text{OPT} \approx 1.15$ versus the LSTM pointer’s ≈ 1.21 — sparser but no better on the trade-off), one training run failed to converge, and we found no improvement justifying the added capacity. We read this as overparametrization relative to the scalar RL signal rather than evidence against attention models in general — consistent with the fact that the probe, a small Transformer trained on dense per-vertex labels, does work; a careful attention-model study is future work.
6. **disc-vis approximation in training.** Both the PO/BT coverage reward and the SETPREDICTOR training targets use disc-vis sampling at $M = 500$. Exact CGAL visibility is used only at evaluation.
7. **Extreme OOD is strong but seed-dependent.** On the large split (Table 7, Section 6.3) the frozen-encoder probe lifts worst-case coverage on every seed while the no-encoder ablation cannot, but the per-seed feasibility count is variable, so we read it as evidence of the encoder signal rather than a feasibility guarantee at this scale; the probe also pays roughly double the local-search guard count, and the vertex-guard optimum is unavailable for the largest polygons there.
8. **Training-curve reconstruction.** Figure 1 is reconstructed from four saved checkpoints (epochs 110, 114, 160, 200), not from an online metric log; per-epoch coverage was not preserved during the original PO/BT run. The trace shows only late-training dynamics, which is sufficient to confirm that the gradient direction does not collapse but does not characterize early-training behavior.

9. Conclusion

We asked what a neural policy internalizes about vertex-guard placement when trained from a coverage-aware reward over its own rollouts and denied any geometric oracle at inference, and we answered it by reading the encoder rather than the decoder: a single-shot probe over the frozen embeddings closes the in-distribution feasibility tail on the displayed seed and compresses the out-of-distribution tail by roughly an order of magnitude across four probe seeds, which we take as evidence that the reinforcement-trained representation already carries more of the geometry than the policy’s decoder revealed. The result is a measurement in a deliberately constrained setting, not a competitor to classical solvers on guard count — the learner is not the best AGP heuristic, but it

arrived at a usable representation of vertex-guard placement without ever being shown the geometry it had to reason about at inference. Two extensions follow naturally: a stronger encoder, such as a Transformer trained under the same PO/BT objective, may shorten the residual tail directly and leave less work for the probe; and a per-instance threshold predicted from the same embeddings would replace the global knob with an adaptive one.

Data and Code Availability

The AGPVG instance library is publicly available [19]. The 313 supplemental `random`-class polygons with their ILP-computed vertex-guard optima, the dataset splits, the trained checkpoints, and all training and evaluation code are available at <https://github.com/dseverdi/AGNet>.

Conflict of Interest

The authors declare no competing interests.

References

- [1] D. T. Lee, A. K. Lin, Computational complexity of art gallery problems, *IEEE Transactions on Information Theory* 32 (2) (1986) 276–282.
- [2] J. O’Rourke, *Art Gallery Theorems and Algorithms*, Oxford University Press, 1987.
- [3] A. Elnagar, L. Lulu, An art gallery-based approach to autonomous robot motion planning in global environments, in: *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2005, pp. 2079–2084. doi: 10.1109/IR0S.2005.1545170.
- [4] B. S. Bigham, S. Dolatikalan, A. Khastan, Minimum landmarks for robot localization in orthogonal environments, *Evol. Intell.* 15 (3) (2022) 2235–2238. doi:10.1007/s12065-021-00616-8. URL <https://doi.org/10.1007/s12065-021-00616-8>
- [5] M. Pan, G. Lin, Y.-W. Luo, B. Zhu, Z. Dai, L. Sun, C. Yuan, Preference optimization for combinatorial optimization problems, in: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2505.08735.
- [6] R. A. Bradley, M. E. Terry, Rank analysis of incomplete block designs: I. the method of paired comparisons, *Biometrika* 39 (3–4) (1952) 324–345.
- [7] R. J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Machine Learning* 8 (3–4) (1992) 229–256.

- [8] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio, Neural combinatorial optimization with reinforcement learning, arXiv preprint arXiv:1611.09940 (2016).
- [9] G. Alain, Y. Bengio, Understanding intermediate layers using linear classifier probes (2017). [arXiv:1610.01644](#).
- [10] J. Hewitt, P. Liang, Designing and interpreting probes with control tasks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 2733–2743.
- [11] Y. Belinkov, Probing classifiers: Promises, shortcomings, and advances, Computational Linguistics 48 (1) (2022) 207–219.
- [12] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 28, 2015.
- [13] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: International Conference on Learning Representations (ICLR), 2019.
- [14] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, S. Min, POMO: Policy optimization with multiple optima for reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020.
- [15] B. Ben-Moshe, M. J. Katz, J. S. B. Mitchell, A constant-factor approximation algorithm for optimal 1.5D terrain guarding, SIAM Journal on Computing 36 (6) (2007) 1631–1647.
- [16] K. Elbassioni, D. Matijević, J. Mestre, D. Ševerdija, Improved approximations for guarding 1.5-dimensional terrains, CoRR abs/0809.0159v1 (2008).
- [17] S. K. Ghosh, Approximation algorithms for art gallery problems in polygons, Discrete Applied Mathematics 158 (6) (2010) 718–722.
- [18] M. C. Couto, P. J. de Rezende, C. C. de Souza, An exact algorithm for minimizing vertex guards on art galleries, International Transactions in Operational Research 18 (4) (2011) 425–448.
- [19] M. C. Couto, P. J. de Rezende, C. C. de Souza, Instances for the Art Gallery Problem, <http://www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery> (2009).
- [20] H. González-Baños, A randomized art-gallery algorithm for sensor placement, Proceedings of the 17th Annual Symposium on Computational Geometry (SoCG) (2001) 232–240.
- [21] Y.-H. Liao, P.-C. Chang, H.-H. Li, Reinforcement learning for optimize coverage in art gallery problem using Q-learning based in grid world, IEEE Access 13 (2025) 52711–52724.

- [22] M. D. Kaba, M. G. Uzunbas, S. N. Lim, A reinforcement learning approach to the view planning problem (2016). [arXiv:1610.06204](#).
- [23] M. Deudon, P. Cournut, A. Lacoste, Y. Adulyasak, L.-M. Rousseau, Learning heuristics for the TSP by policy gradient, in: *Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, Vol. 10848 of *Lecture Notes in Computer Science*, Springer, 2018, pp. 170–181.
- [24] Y. Jin, Y. Ding, X. Pan, K. He, L. Zhao, T. Qin, L. Song, J. Bian, Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem, *aAAI 2023* (2023). [arXiv:2304.09407](#).
- [25] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d’horizon, *European Journal of Operational Research* 290 (2) (2021) 405–421. doi:<https://doi.org/10.1016/j.ejor.2020.07.063>.
URL <https://www.sciencedirect.com/science/article/pii/S0377221720306895>
- [26] I. Mehta, S. Taghipour, S. Saeedi, Pareto frontier approximation network (PA-Net) to solve bi-objective TSP (2022). [arXiv:2203.01298](#).
- [27] K. Li, T. Zhang, R. Wang, Deep reinforcement learning for multiobjective optimization, *IEEE Transactions on Cybernetics* 51 (6) (2021) 3103–3114.
- [28] C. Caramanis, D. Fotakis, A. Kalavasis, V. Kontonis, C. Tzamos, Optimizing solution-samplers for combinatorial problems: The landscape of policy-gradient methods, *neurIPS 2023* (2023). [arXiv:2310.05309](#).
- [29] S. Ross, G. J. Gordon, J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.
- [30] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Computation* 9 (8) (1997) 1735–1780.
- [31] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: *International Conference on Learning Representations (ICLR)*, 2019.
- [32] The CGAL Project, CGAL user and reference manual, <https://doc.cgal.org/5.6/Manual/packages.html> (2024).